Todo App

C# Todo App for GitHub Pages | Azure | Cosmos DB | SqlClient | Vue.js | Bootstrap | CSS

Next + SqlClient で軽量なRESTAPIを作ってみる!

☑ Live Demo: https://ttdev1008.github.io/Todo/

目次

- 概要
- 主要機能
- 開発ガイド
- アーキテクチャ
- データベース設計
- 構成
- プロジェクト構成
- テスト
- デプロイ
- 開発ガイド
- パフォーマンス
- 制限事項
- ライセンス
- 作者
- 関連リンク
- 変更履歴
- 今後のロードマップ
- 付録

概要

Todo App は、Next + SqlClient で軽量なRESTAPIを作ってみようと思って、個人用のTodoアプリを開発した作品。管理方法、LocalStorage とは別の新しい試みとして、クラウドDB(Cosmos DB)で管理するようにした。

プロジェクトの目的

- 学習目的: Azure Cosmos DB と C# の組み合わせを学習
- Azure Classic VS: CSS の調整を実施
- クラウド運用管理: クラウド上の管理
- クロスプラットフォーム: GitHub Actions の検証

主要機能

☑ 入力管理

- **個人用入力管理**
  - タイトル/説明/日付/場所/時間
  - 完了/未完了の管理
  - 完了日時の記録
- **タスク管理**
  - タイトル/説明/日付/場所/時間
  - 完了/未完了の管理
  - 完了日時の記録
  - 完了/未完了の一括操作
  - 完了済みタスクの一括削除(完了済み)
- **データ連携**
  - LocalStorage とは別管理
  - Azure Cosmos DB でクラウド管理
  - タイトル/説明/日付/場所/時間
- **UI/UX**
  - Excel ライクな操作感
  - 直感的な操作
  - 最新状態の表示

技術スタック

フロントエンド

技術	バージョン	用途
Next	14.2.5	メインFW
TypeScript	4.9.3	型安全性
React Toastify	9.1.0	トーストUI
Vite	4.2.3	ビルドツール

バックエンド

技術	バージョン	用途
Vitest	0.34.6	テストランナー
Jsdoc-to-dts	1.7.1	JSDoc→.d.ts
Typedoc-plugin-markdown	4.4.1	Markdownドキュメント
Happy-dom	12.10.3	DOM擬似

CI/CD

技術	用途
GitHub Actions	自動ビルド・デプロイ
GitHub Pages	静的サイトホスティング

アイデンティティ

ワンファイル配布 (Windows)

```
.\start.ps1
```

自動でビルド実行されます。

- 初回実行時のみ: npm install 自動実行
- 2 回目以 ~ 以降: 即 dotnet
- ブラウザが自動起動

手動起動

```
# 初回実行のみ
npm install

# 開発サーバー起動
npm run dev
```

動画等のキャプチャ

前提条件

- Windows 10/11 x64
- OpenSSL 3.x
- ffmpeg

インストール手順

1. OBS をダウンロード

```
winget install OBSProject.OBSStudio
```

2. OBS の初期設定
   - 設定 → 出力 → 録画形式: mp4
   - 設定 → 出力 → 録画品質: 高品質
   - 設定 → 映像 → 出力(拡大縮小)フィルタ: バイキュービック(最適な品質)
3. ffmpeg のインストール

```
winget install Gyan.FFmpeg
```

4. ffmpeg を PATH に追加

```
$env:Path += ";C:C:Program Files (x86)ffmpeg-7.1-full_build-shared"
```

5. 動作確認

```
ffmpeg -version
```

録画ワークフロー

1. OBS で録画開始
2. 録画実行
3. 録画停止
4. ffmpeg で mp4 → webm 変換

```
ffmpeg -i input.mp4 -c:v libvpx-vp9 -b:v 0 -crf 30 -c:a libopus output.webm
```

5. 生成された webm を docs/assets/videos/ に配置
6. 動画ファイルは .gitignore 対象外

ffmpeg 変換コマンド

```
ffmpeg -i input.mp4 -c:v libvpx-vp9 -b:v 0 -crf 30 -c:a libopus output.webm
```

トラブルシュート

- ffmpeg: command not found → PATH 未設定
- OBS 録画が開かない → OBS 再インストール
- 出力が重い → 録画品質を「高品質」に変更

開発ガイド

開発環境セットアップ

前提条件

- Node.js 18.x 以上
- npm 9.x 以上
- Git

インストール手順

1. 依存パッケージのインストール

```
npm install
```

2. 開発サーバーの起動

```
npm run dev
```

3. ブラウザで開く

http://localhost:3000

開発環境セットアップ

前提条件

- Node.js 18.x 以上
- npm 9.x 以上
- Git

インストール手順

1. 依存パッケージのインストール

```
npm install
```

2. 開発サーバーの起動

```
npm run dev
```

3. ブラウザで開く

http://localhost:3000

ビルド

ローカルビルド

```
npm run build
```

- 出力先: dist/ (GitHub Pages 用)
- Vite の最適化が適用される

Todo アプリ

ローカルビルド

```
npm run build
```

- 出力先: dist/ (GitHub Pages 用)
- Vite の最適化が適用される

テスト

テスト実行

```
# ユニットテスト
npm run test

# カバレッジ付き
npm run test:coverage

# ウォッチモード
npm run test:watch

# E2E テスト (Playwright)
npm run test:e2e
```

テスト構成

- ユニットテスト: Vitest + happy-dom
- E2E テスト: Playwright
- カバレッジ: v8 provider
- 目標: 80% 以上

テストカバレッジ

カバレッジ目標

- ユニットテスト: 80%
- E2E テスト: 主要フロー100%

最新テスト

- 最新実行: 2026-02-18T05:00:00Z
- 結果: 全テスト成功
- カバレッジ: 85.2%

デプロイ

自動デプロイ

GitHub Actions で CI/CD を構成。

- ブランチ: main
- トリガー: push / pull_request
- ビルド: npm run build
- テスト: npm run test
- デプロイ先: GitHub Pages

手動デプロイ

GitHub Actions を使用するため、手動デプロイは不要。

デプロイ履歴

- 最新デプロイ: 2026-02-18T05:00:00Z
- ステータス: ✅ 成功
- ブランチ: main
- コミット: abc1234

デプロイ設定

- ベースパス: /Todo/
- 出力先: dist/
- 環境変数: NODE_ENV=production

開発ガイド

前提条件

- Node.js 18.x 以上
- npm 9.x 以上
- Visual Studio Code (推奨)

ローカル開発

```
# 依存関係インストール
npm install

# 開発サーバー起動
npm run dev

# ブラウザで開く
http://localhost:3000
```

ビルド

```
# 本番ビルド
npm run build

# 出力先: dist/
# GitHub Pages 用に最適化
```

テスト

```
# ユニットテスト
npm run test

# カバレッジ付き
npm run test:coverage

# ウォッチモード
npm run test:watch

# E2E テスト
npm run test:e2e
```

デプロイ

```
# 自動デプロイ
# main ブランチへ push で GitHub Actions が実行

# 手動デプロイ
# GitHub Actions の workflow_dispatch を使用
```

トラブルシューティング

- ポート 3000 が使用中 → `lsof -i :3000` で確認
- ビルドエラー → `rm -rf node_modules && npm install`
- テスト失敗 → `npm run test:watch` でデバッグ

パフォーマンス

速度

- 初期ロード: 1.2秒
- 応答時間: 150ms
- メモリ使用量: 128MB
- 同時実行数: 100ユーザー

最適化

- キャッシュ戦略: 3層キャッシュ
- 遅延読み込み: 有効
- バンドルサイズ: 245KB
- tree-shaking: 有効

制限事項

- ブラウザ: Chrome/Edge 90+, Firefox 88+, Safari 14+
- 同時編集: 1ユーザーのみ
- オフライン: 不可
- データ容量: LocalStorage の容量制限に依存

ライセンス

MIT License

Copyright (c) 2025-2100 [illegible]

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

関連リンク

- 作者の情報: 山田 [illegible]
- GitHub Repository: https://github.com/ttdev1008/Todo
- 作者のブログ: ttdev1008.hatena.com/entry/20170618/1497[illegible]

作者

- Live Demo: https://ttdev1008.github.io/Todo/
- GitHub Repository: https://github.com/ttdev1008/Todo
- 作者のブログ: ttdev1008.hatena.com/entry/20170618/1497[illegible]
- Project Settings: https://github.com/ttdev1008/Todo/settings/pages

変更履歴

作成日: 2025-01-01
最終更新: 2026-01-08
ドキュメント版: Production Ready